Google Threat Intelligence App For Microsoft Defender

App Specification Guide

Version 1.0.0

Introduction 3
Microsoft Sentinel vs Microsoft Defender for Endpoint 4
Compatibility Matrix 6
Deployment Architecture 7
- Azure Functions 7
- Google Threat Intelligence 7
- Microsoft Defender 7
- Azure Services 8
- GTI Ingestion App for Microsoft Defender 8

MS Defender App Configuration 9
- Azure Function Configuration 11

Ingested IOCs in Defender 16
Failed to Ingest IOCs 17
Possible Reasons for IOC Rejection from MS Defender 18
Impact of Configuration Changes 19
- Data Ingestion Parameters 19
- Filtering & Scoring 19
- Authentication & Connectivity 19
- System Schedule 19

Configure Microsoft Defender for Endpoint with Microsoft Intune 20
- Pre-requisites 20
- Steps to Configure 20
- Configure MS Defender - Intune 22

Application Logs 24
Debugging Application Logs 26
Limitations/Known Behavior 27

Introduction

This document provides comprehensive Application Specifications for the Google Threat Intelligence (GTI) App integrated with Microsoft Defender. It serves as a detailed guide outlining the installation, setup, and configuration process of the GTI Azure application within the Azure environment.

Microsoft Sentinel vs Microsoft Defender for Endpoint

Microsoft Defender for Endpoint and Microsoft Sentinel serve different but complementary security purposes.

Microsoft Defender for Endpoint (MDE) is primarily used for endpoint protection. It runs directly on devices such as laptops, desktops, and servers to detect, investigate, and respond to threats like malware, ransomware, suspicious processes, or unauthorized access. Security teams use MDE when they want real-time protection and automated response at the device level.

Microsoft Sentinel, on the other hand, is a cloud-native SIEM and SOAR platform. It aggregates and analyzes security data from multiple sources including endpoints, identity systems, cloud services, and network devices. Sentinel is used when organizations need centralized monitoring, correlation of alerts across systems, advanced threat hunting, and automated incident response across the entire environment.

In practice, Defender for Endpoint protects the devices, while Sentinel provides a centralized platform to analyze and investigate security signals from multiple tools (including Defender). Many organizations integrate MDE with Sentinel so that endpoint alerts are ingested into Sentinel for broader security analysis and incident management.

Which GTI Integration to be used

Use Microsoft Defender for Endpoint GTI Integration when:

- You need real-time threat detection and response at the device level
- The focus is on endpoint security (EDR/XDR capabilities)
- You want automated remediation (e.g., isolating a machine, killing processes)
- The use case involves malware detection, ransomware protection, or suspicious process monitoring
- You are primarily dealing with device-specific investigations

Use Microsoft Sentinel GTI Integration when:

- You need centralized visibility across multiple systems
- You want to correlate data from different sources (endpoints, identity, cloud, network)
- The focus is on threat hunting and incident investigation at scale
- You require custom analytics rules and alert correlation
- You want to implement SOAR (automated workflows and playbooks)

  Configuration Guide for Microsoft Sentinel - Link

Compatibility Matrix

Component	**Version / Type**
Google Threat Intelligence	Enterprise Plus or Enterprise and Enterprise Plus licenses
Azure Function Runtime	Java 17
Microsoft Defender for Endpoint	E5 license
Azure Function Plan	Consumption Plan/ Premium Plan
Azure Key Vault	Default
Azure Table Storage	NoSQL Key-Value Store

Deployment Architecture

Azure Functions

Azure Functions serves as the serverless compute backbone of this solution, providing an event-driven environment to execute the data transformation logic. By utilizing the **Consumption** model, the solution achieves high-throughput scaling and high-performance execution without the overhead of traditional server management. This architecture ensures that the data pipeline remains highly available and cost-effective, automatically scaling resources up or down based on the volume of threat intelligence data being processed from GTI. The consumption model allows for refined control over instance memory and concurrency, ensuring the transformation of complex threat telemetry is handled efficiently.

Google Threat Intelligence

Google Threat Intelligence is a unified security solution that synthesizes the elite research of Mandiant, the massive crowdsourced malware database of VirusTotal, and Google's global visibility into internet traffic. By providing a comprehensive view of the global threat landscape, GTI empowers organizations to stay ahead of adversaries. In this deployment, GTI acts as the primary data source, providing raw, high-fidelity threat indicators and contextual telemetry that are essential for proactive defense.

Microsoft Defender

Microsoft Defender is an enterprise endpoint security platform designed to help organization networks prevent, detect, investigate, and respond to advanced threats. It serves as the destination for this integration, where transformed intelligence is ingested as custom indicators or alerts. By feeding enriched data from GTI into Defender, security teams can automate blocklists, trigger automated investigations, and gain a centralized view of specialized threats within their primary XDR (Extended Detection and Response) console.

Azure Services

- **Azure Key Vault:** Securely manages API keys for GTI and sensitive credentials, ensuring they are never hard-coded and are accessed only by the authorized Function identity.

- **Azure Storage:** Provides the persistent backend for function state management, internal queuing, and trigger synchronization.

- **Application Insights:** Offers deep observability, monitoring the health of the ingestion pipeline and providing real-time telemetry on transformation success rates and execution bottlenecks.

- **Azure Function:** Serves as the core compute layer that orchestrates the ingestion pipeline. It is responsible for fetching threat intelligence data from GTI APIs, transforming it into the required schema, and ingesting it into the target platform. It supports event-driven and scheduled execution (via timers), scales automatically based on workload, and integrates securely with other Azure services such as Key Vault, Storage, and Application Insights to ensure reliable, scalable, and maintainable processing.

GTI Ingestion App for Microsoft Defender

As the threat landscape becomes increasingly complex, the speed at which external intelligence is converted into internal protection is a key metric for SOC efficiency. The **GTI Ingestion App** automates the bridge between Google's world-class threat data and Microsoft's defensive ecosystem.

The application retrieves indicators from GTI, performs complex data transformation to align with Microsoft's schema, and pushes these insights into Defender. This automated pipeline removes the manual burden of "copy-pasting" IOCs between platforms, ensuring that your defensive perimeter is updated in near real-time. By shortening the time-to-detect and time-to-remediate, this integration allows security practitioners to focus on high-level analysis while the automated system hardens the environment against the latest global threats.

MS Defender App Configuration

Prerequisites

- Valid GTI API key.
- Global Security Administrator role (Microsoft)
- Current user object id (steps shown below)
- ClientID, Client Secret, ApplicationID through App registration(steps shown below)

How to get current user object id:

- Sign in: Go to the Microsoft Entra admin center.
- Navigate to Users: In the left-hand sidebar, expand Identity, then select Users > All users.
- Search for the User: Use the search bar to find the specific user (e.g., yourself).
- Open Profile: Click on the Display Name of the user.
- Copy the ID: Under the Overview or Basic info section, locate the field labeled Object ID.

App Registration

Step 1: Create the App Registration

- Sign in to the Microsoft Entra admin center as a Security Administrator or Global Administrator.
- Expand Identity in the left menu and select Applications > App registrations.
- Click + New registration.
- Enter a descriptive Name.
- Under Supported account types, select Accounts in this organizational directory only (Single tenant).
- Leave the Redirect URI blank and click Register.

Step 2: Collect Application (Client) and Directory (Tenant) IDs

Once the app is created, you will be taken to the Overview page.
Locate and copy the following values to your secure notes/Key Vault:

- Application (client) ID - This is the ClientID
- Directory (tenant) ID - This is the TenantID

Step 3: Generate a Client Secret

- In the left sidebar of your app registration, select Certificates & secrets.
- Select the Client secrets tab and click + New client secret.
- Add a Description and select an Expires duration (standard is 180 days).
- Click Add.

CRITICAL: Copy the Value of the secret immediately. You cannot see this value again after you leave this screen.

Step 4: Add Permissions for Microsoft Defender

Based on the official documentation for the import-ti-indicators API, you must add the "Application" permission type so the service can run without a signed-in user.

- In the left sidebar, select API permissions.
- Click + Add a permission.
- Select the APIs my organization uses tab.
- In the search bar, type WindowsDefenderATP and select it.
- Choose Application permissions.
- In the search box, find and check:
- Ti.ReadWrite.All (Read and write All Indicators)
- Click Add permissions.
- Choose Delegated permissions
- In the search box, find and check:
- Ti.ReadWrite
- Click Add permissions.

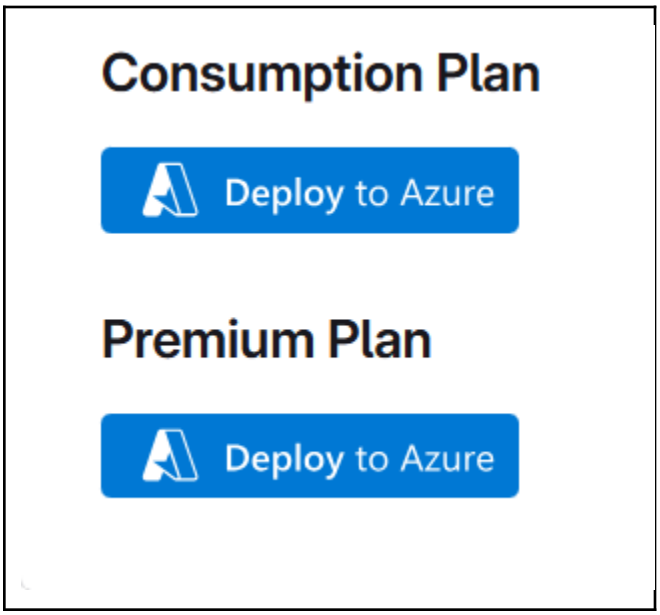
Step 5: Grant Admin Consent

- Adding the permission is not enough; a high-level admin must authorize it for the tenant.
- On the same API permissions screen, look for the button that says Grant admin consent.
- Click it and select Yes to confirm.
- The status for Ti.ReadWrite.All and Ti.ReadWrite should now show a green checkmark under the "Status" column.

For detailed steps follow Microsoft Documentation: link

Azure Function Configuration

- Go to <>
- Choose an appropriate plan. For more help deciding to choose which plan to go for refer:
    - Azure function pricing
    - Azure function pricing calculator



- You will be redirected to azure portal. Where you will see the below Azure Deployment Form.

Customized template
undefined resources

Edit template | Edit parameters | Visualize

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * : CDS_R15_Sub1

Resource group * :

Create new

Instance details

Region * : East US

App Name * :

Gti Api Key * :

Threat Lists :

Look Back Days : 7

Verdicts :

Severities :

Threat Score :

Timer Schedule : 0 * */1 * * *

Current User Object Id * :

App Client ID * :

App Client Secret * :

App Tenant Id * :

- Complete the Azure Deployment Form using the following

Parameter	Description	Requirement	Format / Default
Subscription	Azure Subscription for billing.	**Mandatory**	Dropdown selection
Resource Group	Logical container for resources.	**Mandatory**	New or Existing
Region	Physical data center location.	**Mandatory**	e.g., West Europe
App Name	Unique name for your Function App.	**Mandatory**	String
GTI API Key	Token for Google Threat Intelligence.	**Mandatory**	Secure String
Threat List	Specific GTI list IDs to monitor.	*Optional*	IDs (Empty = All)
Look Back Days	Defines the number of past days to retrieve data for during the initial ingestion,	**Mandatory**	Integer (**Max 7**)

	enabling backfill of historical threat intelligence.		
Verdicts	Filter by GTI verdict levels.	*Optional*	Comma-separated. Empty = All
Severities	Filter by GTI severity levels.	*Optional*	Comma-separated. Empty = All
Threat Score	Minimum score for ingestion.	**Mandatory**	Integer (**0–100**)
Timer Schedule	Frequency of data fetch.	**Mandatory**	6-field CRON
User Object ID	Your ID for Key Vault access.	**Mandatory**	GUID / Object ID
Client ID	"Application ID" from App Registration.	**Mandatory**	GUID
Client Secret	"Secret Value" from App Registration.	**Mandatory**	Secure String

Tenant ID	"Directory ID" from Entra ID.	**Mandatory**	GUID

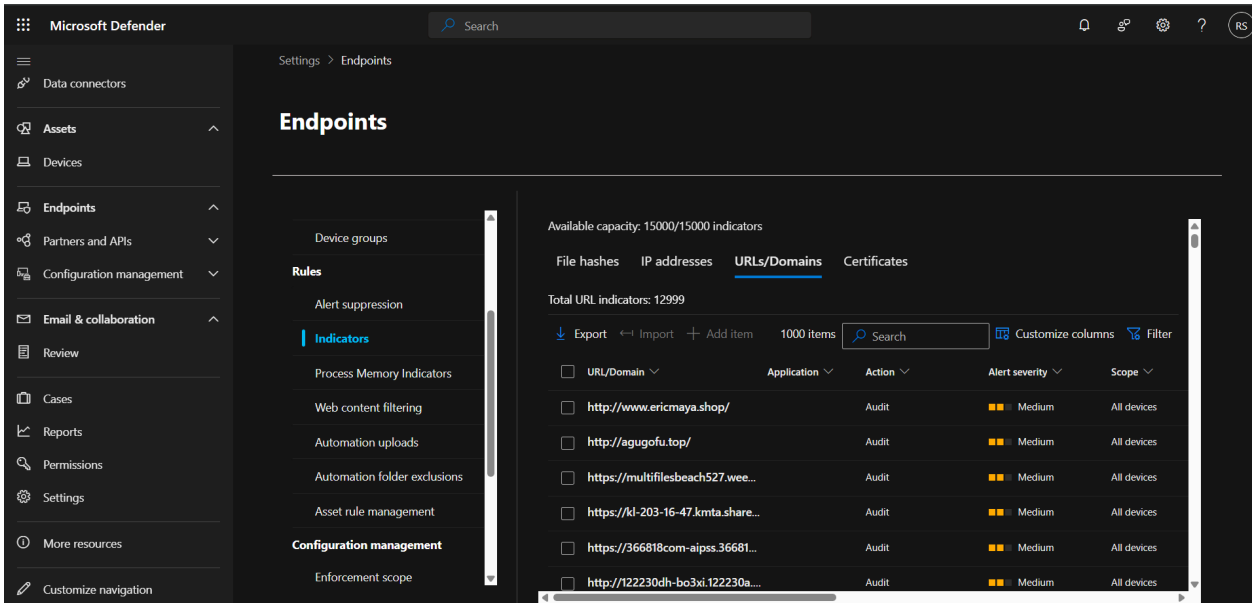
- Click Review + Create and then check for the values you have entered.
- Click create and soon deployment will start.

Note: You can change the above parameters from Application Settings once the deployment is complete.

Ingested IOCs in Defender

You will require global security Administrator role to see the ingested IOCs

- Navigate to security.microsoft.com.
- In the left pane go to settings > endpoints.
- In the inner left pane go to rules > indicators. You will see something like this



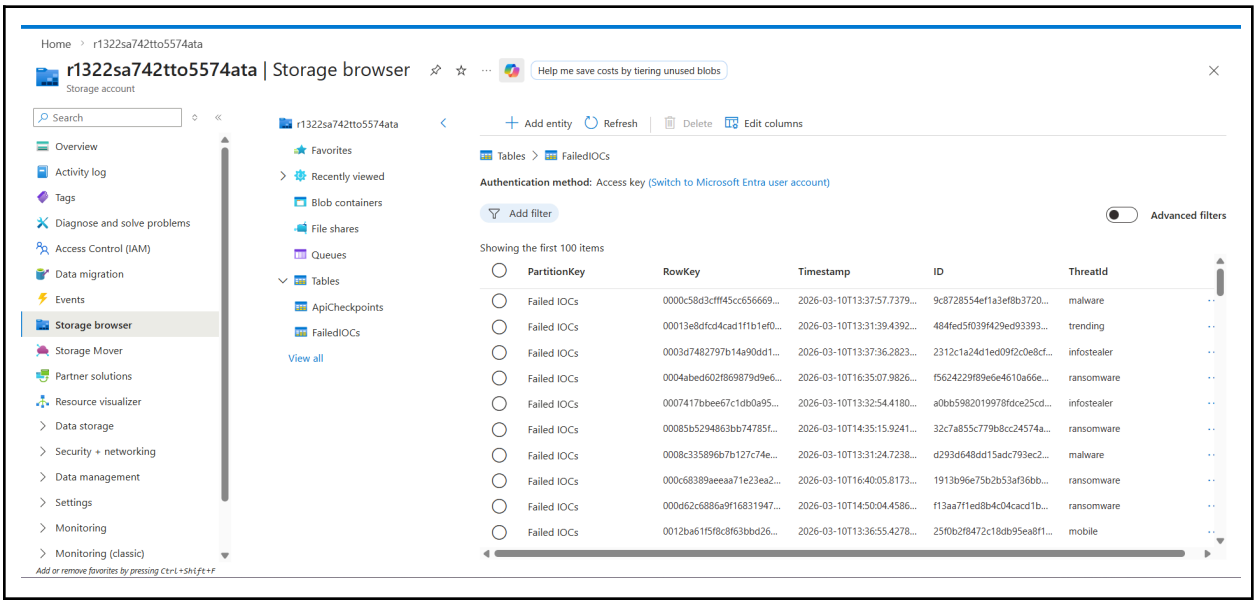
- Here you will see all the ingested IOCs in Defender.

Failed to Ingest IOCs

It is possible that some IOCs fail to ingest into Microsoft Defender, as Defender may reject certain indicators. In such cases, the failed IOCs can be viewed in Azure Table Storage by following these steps:

- Go to the storage account created by the deployment script.
- In the left pane, navigate to Storage Browser.
- Select Tables → FailedIOCs.

Here, you will be able to see the list of IOCs that failed to ingest.



Possible Reasons for IOC Rejection from MS Defender

Microsoft Defender may reject the IOC for ingestion due to the following reasons

- **Defender Limit Exhaustion:** MS Defender has a 15000 limit of custom indicators. Once the limit is reached Defender will reject ingestion request
- **Invalid File Hash:** While trying to ingest Indicator of type File. It may happen that MS Defender rejects the file hash that is fetched from GTI resulting in ingestion failure.
- **Invalid Expiration Time:** If the expiration time is in the past, MS Defender will reject that particular Indicator.
- **Invalid IP Address:** If the IP Address mentioned in the indicator is internal, MS Defender will reject the indicator.

Impact of Configuration Changes

Changing these parameters in **Application Settings / Azure Key Vault** triggers specific behaviors during the next function execution.

Data Ingestion Parameters

- **Threat Lists:** Adding a new list triggers a fresh fetch based on your **Lookback Days**.
    - *Example:* If Lookback is 2 days and the app runs at 2 PM on Feb 20, it retrieves data starting from 2 PM on Feb 18 for that list.
- **Lookback Days:** Defines the historical window (Maximum: **7 days**).
    - **Initial Run:** Fetches data from X days ago until now.
    - **Checkpoint Override:** If a stored checkpoint is older than the Lookback window, the app automatically resets the checkpoint to `Today - Lookback Days` to prevent stale data ingestion.

Filtering & Scoring

- **Verdicts, Severities, and Threat Score:** Changes apply to the **next batch** of data retrieved via the checkpoint.
    - **Note:** These filters may affect "past" data if that data is part of the next scheduled sync.
    - **Threat Score Constraint:** Must be an integer between **0 and 100**.

Authentication & Connectivity

- **GTI API Token:** The new token is validated and used immediately upon the next execution.
- **Client ID, Client Secret, and Tenant ID:** Updated credentials apply to the next ingestion cycle to authenticate with Microsoft Defender.

System Schedule

- **Timer Schedule (CRON):** Changing this triggers an **automatic restart** of the Function App to re-register the schedule.
    - **Recommendation:** Use **hourly intervals** for optimal performance.

Configure Microsoft Defender for Endpoint with Microsoft Intune

Pre-requisites

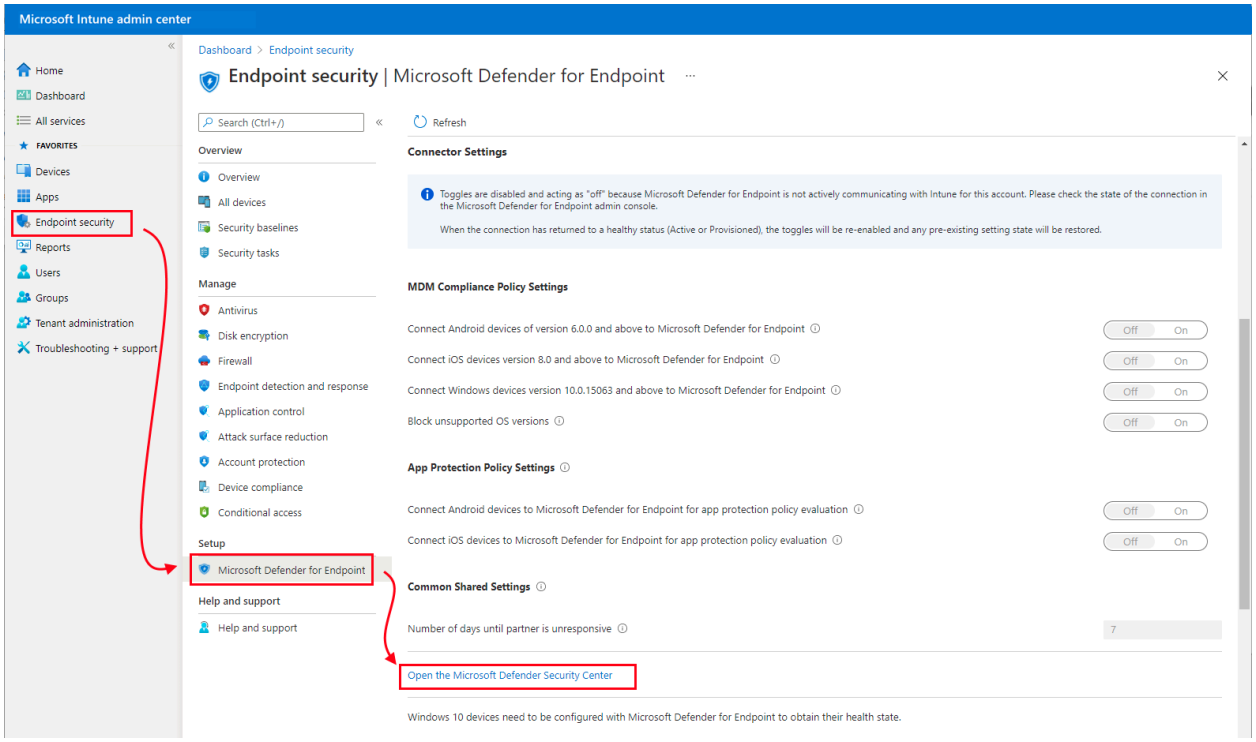
- Admin access to the Microsoft Intune admin center with Endpoint Security Manager role or equivalent permissions for Mobile Threat Defense settings (custom roles require Read and Modify rights for the Mobile Threat Defense permission).
- Admin access to the Microsoft Defender XDR portal with Security Administrator role in Microsoft Entra ID, or "Manage security settings in Security Center" permission in Microsoft Defender for Endpoint.

Steps to Configure

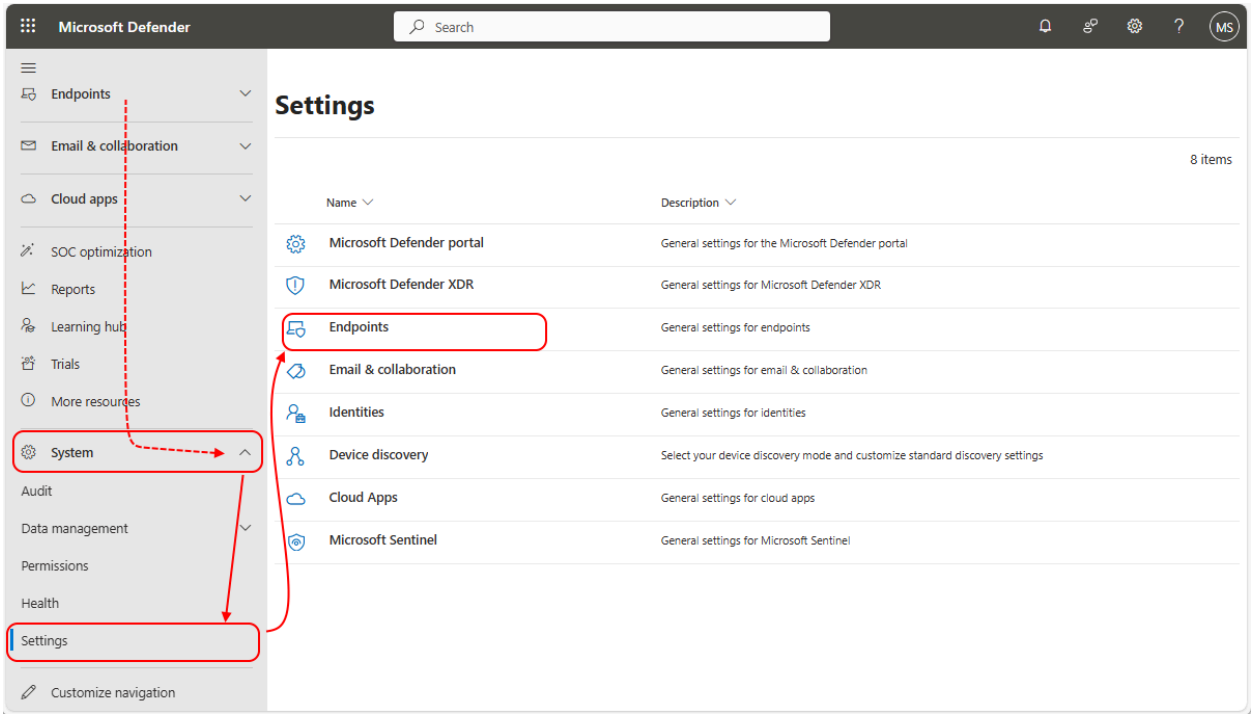
- Check connection status first: Sign in to the Microsoft Intune admin center and select Endpoint security > Microsoft Defender for Endpoint.

    - If Connection status shows Enabled, the services are already connected. Skip to For app protection policy evaluation.
    - If Connection status shows Unavailable, continue with the next step.

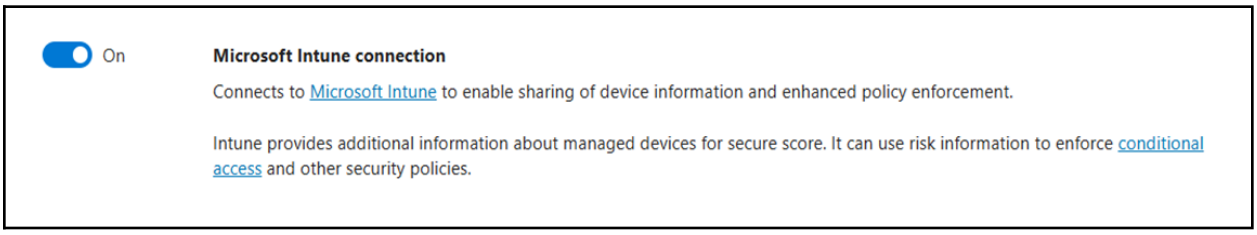
- Open Microsoft Defender XDR portal: From the Intune admin center, scroll to the bottom of the Microsoft Defender for Endpoint page and select Open the Microsoft Defender Security Center



- Enable the connection in Microsoft Defender portal:
  - Navigate to Settings > Endpoints > General > Advanced features.



- Locate Microsoft Intune connection and toggle it to On.



- Select Save preferences.

- **Validation:** Return to the Intune admin center. The Connection status should now show Enabled (it can take up to 15 minutes to update). Services sync automatically every 24 hours, and you can adjust monitoring settings under Endpoint security > Microsoft Defender for Endpoint if needed.

Configure MS Defender - Intune

What this accomplishes

- Choose which device platforms connect to Microsoft Defender for Endpoint and enable specific integration features like compliance evaluation and app protection policies via Intune.

Configuration

- **Navigate to integration settings**: In the Microsoft Intune admin center, go to Endpoint security > Microsoft Defender for Endpoint. The Connection status should now show Enabled.
- **Configure compliance policy evaluation:** Enable these options under Compliance policy evaluation for your supported platforms:
  - Connect Android devices to Microsoft Defender for Endpoint: On
  - Connect iOS/iPadOS devices to Microsoft Defender for Endpoint: On
  - Connect Windows devices to Microsoft Defender for Endpoint: On
- **Configure app protection policy evaluation:** Enable these options under App protection policy evaluation for mobile platforms:
  - Connect Android devices to Microsoft Defender for Endpoint: On
  - Connect iOS/iPadOS devices to Microsoft Defender for Endpoint: On

- **Save your configuration:** Select Save to apply all settings.

Your integration is configured! The platforms you enabled will now connect devices to Microsoft Defender for Endpoint for threat assessment and compliance evaluation.

For more information about integrating Microsoft Defender to Microsoft Intune refer: link

Application Logs

To view application logs:

- Open your Function App resource.
- In the left-hand menu, click on **Monitoring.**
- Click on **Logs** under the Monitoring section.

Running Queries to View Past Logs

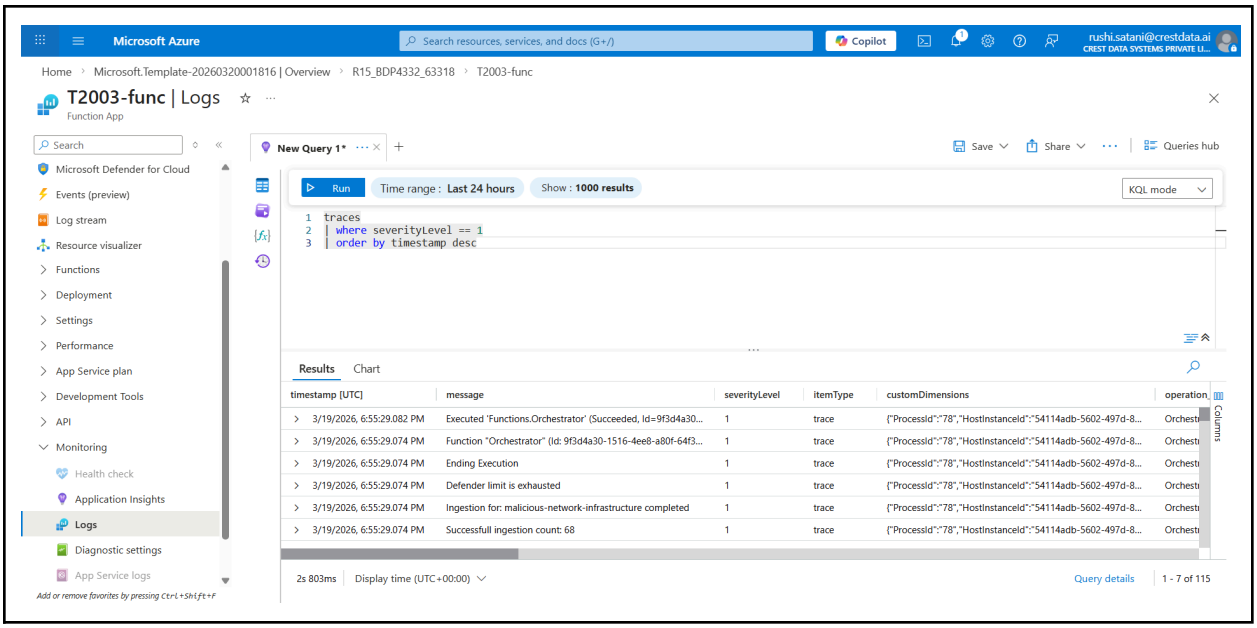
Once inside the Logs section, you can use **Kusto Query Language (KQL)** to retrieve historical data.

View Function Execution Logs

To view function execution logs you can use the following query

```
None
traces
| where severityLevel == 1
| order by timestamp desc
```

This will give you all the historical logs as shown below


Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
rushi.satani@crestdata.ai
CREST DATA SYSTEMS PRIVATE LI...
Home > Microsoft.Template-20260320001816 | Overview > R15_BDP4332_63318 > T2003-func
T2003-func | Logs
Function App
Search
Microsoft Defender for Cloud
Events (preview)
Log stream
Resource visualizer
Functions
Deployment
Settings
Performance
App Service plan
Development Tools
API
Monitoring
Health check
Application Insights
Logs
Diagnostic settings
App Service logs
Add or remove favorites by pressing Ctrl+Shift+F
New Query 1*
Save
Share
Queries hub
Run
Time range : Last 24 hours
Show : 1000 results
KQL mode
traces
| where severityLevel == 1
| order by timestamp desc
Results
Chart
timestamp [UTC]
message
severityLevel
itemType
customDimensions
operation_
3/19/2026, 6:55:29.082 PM
Executed 'Functions.Orchestrator' (Succeeded, Id=9f3d4a30...
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
3/19/2026, 6:55:29.074 PM
Function "Orchestrator" (Id: 9f3d4a30-1516-4ee8-a80f-64f3...
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
3/19/2026, 6:55:29.074 PM
Ending Execution
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
3/19/2026, 6:55:29.074 PM
Defender limit is exhausted
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
3/19/2026, 6:55:29.074 PM
Ingestion for: malicious-network-infrastructure completed
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
3/19/2026, 6:55:29.074 PM
Successfull ingestion count: 68
1
trace
{"ProcessId":"78","HostInstanceId":"54114adb-5602-497d-8...
Columns
2s 803ms
Display time (UTC+00:00)
Query details
1 - 7 of 115

Debugging Application Logs

The application logs key messages in **Azure Application Insights** to help identify issues during ingestion:

- **Invalid GTI API key, please change it from Azure Key Vault**
  The configured GTI API key is invalid or expired. Update it in Azure Key Vault.
- **Either of the following is invalid: Client id, Client Secret, Application ID. Verify and change from Azure Key Vault**
  One or more Microsoft Defender authentication values are incorrect or expired. Verify and update the credentials in Azure Key Vault.
- **Defender limit is exhausted**
  The maximum limit of ~15,000 indicators in Microsoft Defender has been reached. Remove old indicators or reduce ingestion.
- **IOCs failed to ingest:** <count>
  Number of indicators that failed in the batch due to validation errors, duplicates, or API constraints.
- **Successful ingestion count:** <count>
  Number of indicators successfully ingested in the batch.

Limitations/Known Behavior

- Microsoft defender has a maximum capacity of 15000 indicators to be stored at any given point in time. Although we make sure to expire them, it may be possible to miss out on some of them.
- The Microsoft Defender Import API may not return the exact same number of indicators in the response as were included in the request. This is because each indicator is processed individually, and some may be rejected, skipped, or deduplicated based on validation rules, existing data, or policy constraints within Microsoft Defender. As a result, the response typically reflects only the indicators that were successfully processed, rather than a one-to-one mapping of the request payload.
- In the consumption plan there is a limit of 10 min for the execution of the function, after which the function gets terminated itself. In that case the function will continue ingesting the remaining data in the next execution.
- If the defender limit is exceeded during an execution, the checkpoint will not be updated. As a result, when the next execution resumes it will utilize the same timestamp, and the same data will be processed again to prevent any data loss.

—----------End of Document—----------